

Python Programming Assignment - 5

1) What are bytes in Python?
Why are they immutable?

Ans: Bytes datatype represents an immutable sequence of bytes mostly used in file handling and networking.

- Immutable sequence of bytes
- Values range from 0 to 255
- Used in file handling and networking.

They are immutable because it prevents your files, images or network data from getting corrupted or accidentally ruined. It represents raw binary data (like images, files, or network data packets).

2) Predict the output:

```
b = bytes([65, 66, 67])  
print(b)
```

Explain how numbers are converted internally?

Ans: Output:
`b'ABC'`

Internally, Python converts these numbers using the ASCII character chart.

Every number you pass into the `bytes()` function represents a specific English letter or symbol because

65 is code for capital A

66 is code for capital B

67 is code for capital C

When we run `print(b)`, Python displays a small `b` in front of quotes (`b'...'`) to show the data is stored as raw binary bytes, followed by the next text characters it calculated from numbers.

3) What is the difference between bytes and bytearray? Mention mutability and use cases.

Ans: The difference between bytes and bytearray objects is mutability. Both are used to store sequence of single bytes (integers in the range $0 \leq x < 256$), but they handle modifications differently.

- bytes: These are immutable. Once a bytes object is created, you can change, add, or remove its individual elements. If we modify it will actually create a new bytes object in memory.
- bytearray: These are mutable. You can modify, append, insert or delete elements in place after the object has been created, much like a standard Python list.

Use cases:

i) When to use bytes:

- Fixed Binary Data: When reading binary data from a file that you only need to parse or analyze, but not alter.
- Data Integrity: When you want to guarantee that a sequence of bytes remains constant and safe from accidental modification during program execution.
- Dictionary keys: Because they are immutable and hashable, bytes objects can be used as keys in dictionaries or stored in sets.

ii) When to use bytearray:

- In-Place Modification: When performing operations like altering file headers where you need to change specific bytes without creating entirely new copies of large datasets in memory.

- Data Buffering: When receiving streamed data over a network in chunks and assembling in byte-by-byte, or when binary payload dynamically before sending it.

4) Predict the Output:

```
ba = bytearray([65, 66, 67])
```

```
ba[0] = 97
```

```
print(ba)
```

Why is this allowed?

Ans: Output:

```
bytearray(b'aBC')
```

This is allowed because bytearray objects in Python are mutable.

i) `ba = bytearray([65, 66, 67])`: This creates a bytearray with three elements in ASCII, 65 represents 'A', 66 represents 'B', and 67 represents 'C'.

ii) `ba[0] = 97`: Because bytearray is mutable, you can reassign values at specific indices. This line changes the integer at index 0 from 65 to 97. In ASCII, 97 represents the lowercase letter 'a'.

iii) `print(ba)`: When printed, Python displays the bytearray representation.

5) What is None in Python?

Is it same as 0, False, or empty string?

Ans: None is a Data type which represents the absence of a value and is commonly used as a placeholder or default return value.

No, they are not same as 0, False or empty strings because None means the value doesn't exist but the memory is allocated by the RAM.

6) Predict the output:

`x = None`

`print (type(x))`

`print (x == False)`

Explain the result.

Ans: Output:

`<class 'NoneType'>`

`False`

Explanation -

Since, user has defined or declared that `x` is `None` therefore type of `x` will be `NoneType` because `x` is not declared with any kind of other object such as integer, string, etc. It is a special kind of object which is used to just allocate the memory without any value stored in it.

The `==` operator checks if the value are equal or not. Because `None` means 'no value exists' and `False` is an actual boolean type value not equal. Therefore, nothing is equal to `False` results in the answer 'False'.

7) What is the Boolean data type in Python? List all values that evaluates to False.

Ans: The Boolean (bool) datatype represents truth values, mainly used in decision making and conditional statements.

`True` is internally stores as 1 and `False` is stored as 0.

Any function or datatype which is declared but its value is zero will evaluates to `False`. Also an empty sequences and collections results to 'False'.

Below is the example of Lists :-

a) For any numeric type -

- 0 (integer value)
- 0.0 (float value)
- 0j (complex value)
- Decimal(0) (from Decimal module)

b) For any empty sequences and collections

- "" or '' (Empty strings)
- [] (Empty list)
- () (Empty tuple)
- {} (Empty dictionary)
- set() (Empty set)
- range(0) (Empty range), etc.

Conclusion - Nothing is False. Something is True.

If anything inside of it, or a number has any value attached to it is considered as True. And if it is empty, blank or zero, it is considered as False.

8) What is a set in Python?

Ans: A set is an unordered list of collection used to store unique elements only, mainly for removing duplicates. The set is mutable and is also used for removing duplicates values.

ex:

```
S = {10, 20, 30, 20}
```

```
print(S)
```

```
# {10, 20, 30}
```

```
print(type(S))
```

```
# <class 'set'>
```

9) Predict the output:

$S = \{10, 20, 30, 10, 20\}$

print(s)

Why are some values missing?

Ans: Output:

$S = \{10, 20, 30\}$

Set has mainly characteristics that it holds unique values elements. Also it automatically removes duplicate values or elements which is stored in the set and gives us the output. It accepts first 10 & 20, but the second 10 & 20 are duplicates therefore it automatically removes them.

10) What is dynamic typing in Python?

Ans: Dynamic typing in Python means the users do not have to explicitly declare a data type to the variable. Python automatically detects its data type based on the value assigned to it at runtime. The interpreter figures out the type on the fly based on whatever value happens to be sitting inside at that runtime.